

# SMGO Implementation Guide

A line-by-line technical walkthrough of the features we implemented so you can explain, reproduce, or defend the design in questions or interviews.

## 1. What was implemented

| Feature                     | What it does                                                                                        |
|-----------------------------|-----------------------------------------------------------------------------------------------------|
| Top 10 Content              | Exposes a dedicated backend <code>/api/contents/top10</code> endpoint and wires the frontend to it. |
| Multi-channel notifications | Creates in-app notifications immediately and sends an email fallback after a delay if unread.       |
| AI recommendations          | Uses a real Python XGBoost service with joblib persistence and a Java-to-Python bridge.             |

## 2. Multi-channel notifications

### Data model

- The notification entity now stores fallback timing fields: `emailFallbackDueAt`, `emailFallbackSentAt`, and a boolean `emailFallbackSent` flag.
- That means the system knows when to send fallback email and whether it already did it.

### Repository query

- A Mongo repository method finds unread notifications where fallback was not sent and the due time has already passed.
- That is the scheduler's selection set.

### Service flow

- `createNotification` saves the record with `isRead=false` and `emailFallbackDueAt = now + configured delay`.
- `markAsRead` sets `emailFallbackSent=true` so a later fallback run will ignore already-read notifications.
- `sendEmail` uses `JavaMailSender` and `SimpleMailMessage` to send a real SMTP email.
- A scheduled method runs every minute and reuses the same fallback processor.

### Manual test hook

- The `/api/notifications/test/trigger-email-fallback` endpoint lets you force the fallback processor immediately for testing.

### Configuration

- `application.properties` stores the SMTP host, Gmail username, app password placeholder, sender address, and delay settings.

## 3. Top 10 content

- The service contract adds `getTop10Content(category, genreKeyword)`.
- The controller exposes `GET /api/contents/top10`.
- The implementation reuses the analytics repository query and hard-limits it to 10 items.
- The frontend admin advanced page now calls the dedicated endpoint instead of sorting locally only.

## 4. XGBoost AI recommender

- `train_model.py` creates a messy dataset with duplicates and missing values, then cleans it using `dropna()` and `drop_duplicates()`.

- The cleaned data is split into train and test sets, one-hot encoded for categorical fields, and trained with XGBRegressor.
- The trained pipeline is saved with joblib to ai-service/model/content\_recommender.joblib.
- app.py loads the saved model; if loading fails, it retrains automatically.
- The /recommend endpoint builds a feature row from the user profile and each content item, predicts scores, and returns ranked recommendations.
- The Java backend calls the Flask /recommend service from ContentServiceImpl and falls back to local heuristic scoring if the service is unavailable.

## 5. Key code paths

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| Backend content API | ContentController -> ContentService -> ContentServiceImpl -> Python AI service      |
| Notification flow   | NotificationController -> NotificationServiceImpl -> NotificationRepository -> SMTP |
| AI training flow    | train_model.py -> joblib model -> app.py -> /recommend                              |

## 6. How to explain it in an interview

- The top 10 feature is now a real API backed by analytics, not just a frontend sort.
- The notification system is multi-channel: in-app immediately, email later if unread.
- The AI recommender is truly trained, not hard-coded. It learns from synthetic interaction labels and predicts ranking scores.
- The Java backend is resilient because it can fall back to local recommendation logic if the Python service is down.
- The data pipeline is reproducible because it generates messy data, cleans it, trains, and stores the model consistently.

## 7. Validation we already ran

- Backend compile succeeded with Maven.
- The Flask AI service health endpoint returned status ok and metrics.
- The Python sample client returned ranked recommendations from the XGBoost model.
- The Java recommendation endpoint returned AI-backed results from the running backend.

## 8. Best next improvement

If you want the recommender to be stronger, the best upgrade is a hybrid model: keep XGBoost for ranking, add collaborative signals from reservations/watch activity, and include time-based or session-based context. That is better than category/type alone.

## 9. Scheduled Newsletter Campaign

- This is a separate campaign system, not the unread-email fallback notification flow.
- A newsletter campaign is stored in its own Mongo collection and exposed through /api/newsletters.
- When the scheduled time arrives, the service creates in-app notifications directly and sends email immediately if enabled.
- Because the newsletter notifications are saved with emailFallbackSent=true, they do not enter the delayed unread-email fallback logic.
- The admin notifications page now includes a newsletter scheduling modal and a campaign table.
- Audience targeting can be all users, a content category, or genre names inferred from reservation history.

### Newsletter flow

- Create campaign -> store scheduledAt and filters
- Scheduler finds due campaigns

- Service resolves target users
- Create in-app notifications directly
- Send SMTP email immediately if enabled

## 10. Line-reference appendix

| File                                                                                                | Important lines       | What to say                                                                      |
|-----------------------------------------------------------------------------------------------------|-----------------------|----------------------------------------------------------------------------------|
| backend/src/main/java/com/example/contentmanagement/service/impl/NotificationServiceImpl.java       | 29-41, 54-61, 106-110 | This ServiceImpl, fallback scheduling, and mark-as-read protection happens here. |
| backend/src/main/java/com/example/contentmanagement/controller/NotificationController.java          | 36-40                 | This is the manual trigger endpoint for fallback testing.                        |
| backend/src/main/java/com/example/contentmanagement/entity/Notification.java                        | 38-41                 | These fields track when fallback should happen and whether it already happened.  |
| backend/src/main/java/com/example/contentmanagement/controller/ContentController.java               | 13-15                 | This is an explicit Top 10 API route.                                            |
| backend/src/main/java/com/example/contentmanagement/service/impl/ContentServiceImpl.java            | 50-51, 57, 139-140    | This class calls the Python recommender and falls back if needed.                |
| backend/src/main/java/com/example/contentmanagement/service/impl/NewsletterCampaignServiceImpl.java | 38-41                 | This class schedules newsletter campaign flow and dispatcher.                    |
| backend/src/main/java/com/example/contentmanagement/controller/NewsletterCampaignController.java    | 16-15                 | This is the Newsletter Campaign Controller API used by the admin page.           |
| ai-service/train_model.py                                                                           | L65-L236              | This is the XGBoost training pipeline and model save path.                       |
| ai-service/app.py                                                                                   | L19-L28, L144-L192    | This is the Flask inference service and live recommendation API.                 |

## Important files

- backend/src/main/java/com/example/contentmanagement/service/impl/ContentServiceImpl.java
- backend/src/main/java/com/example/contentmanagement/controller/ContentController.java
- backend/src/main/java/com/example/contentmanagement/service/impl/NotificationServiceImpl.java
- backend/src/main/java/com/example/contentmanagement/controller/NotificationController.java
- ai-service/train\_model.py
- ai-service/app.py
- new features.md